Gabriele Cerioli
"Laboratorio di calcolo"

Agosto 2024

Ricerca del massimo autovalore di una matrice simmetrica

Indice

1 Problema da risolvere **2**

2 Breve introduzione teorica **2**
2.1 Dimostrazione della prima formula 2
2.2 Dimostrazione della seconda formula 2

3 Implementazione in linguaggio Fortran delle formule **3**
3.1 Potenza di una matrice 3
3.2 Calcolo della traccia di una matrice 3
3.3 Algoritmo di controllo che la matrice sia simmetrica 3
3.4 Implementazione delle due formule 4

4 Analisi del comportamento in funzione di N **4**
4.1 Comportamento dello scarto rispetto al valore atteso 5

5 Comportamento per N ancora maggiori, e possibilità di scelta dei kind **6**
5.1 Comportamento dell'errore relativo tra iterazioni successive 7

6 Approfondimenti **10**
6.1 Condizione d'uscita 10
6.2 Casi particolari e limiti di funzionamento dell'algoritmo 11
6.3 Problemi dell'algoritmo 12

1 Problema da risolvere

Due formule per il calcolo del massimo autovalore λ_1 di una matrice simmetrica A sono:

$$|\lambda_1| = \lim_{n\to\infty} (Tr(A^n))^{\frac{1.0}{n}}$$

$$\lambda_1 = \lim_{n\to\infty} \frac{Tr(A^{n+1})}{Tr(A^n)}$$

Implementarle e studiarne il comportamento in funzione di n nel caso della matrice simmetrica A, inoltre discuterne anche gli aspetti di precisione del calcolo.

$$A = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

2 Breve introduzione teorica

Il *teorema spettrale*, studiato nel corso di Geometria, ci assicura che la matrice $A \in M_{M\times M}$ simmetrica sia ortogonalmente diagonalizzabile, dunque che sia scrivibile come $A = PDP^T$, con P matrice ortogonale e D matrice diagonale che contiene tutti gli autovalori. **Ci assicura dunque che abbia m autovalori reali. Questo ci permetterà di utilizzare una variabile di tipo *real* nell'implementazione.**

2.1 Dimostrazione della prima formula

Notiamo che 1.0 all'esponente è presente solo ai fini dell'implementazione in Fortran, dunque per gli obiettivi della dimostrazione possiamo considerarlo equivalente a 1. Per il teorema spettrale vale $A = PDP^T$, e sulla diagonale di D ci sono gli autovalori di A. Inoltre

$$Tr(A^n) = Tr(PD^nP^T) = Tr(D^n) = \lambda_1^n + \cdots + \lambda_m^n$$

dove la prima uguaglianza deriva dalla proprietà associativa del prodotto tra matrici. Mentre la seconda si può dimostrare brevemente in questo modo, essendo P ortogonale $Tr(PAP^T) = Tr(PAP^{-1})$ e

$$Tr(PAP^{-1}) = Tr(AP^{-1}P) = Tr(AI_M) = Tr(A)$$

dove è stata utilizzato il fatto che la traccia è invariante rispetto ad una permutazione ciclica.
Inoltre, supponendo senza perdere di generalità che λ_1 sia l'autovalore massimo in valore assoluto, :

$$\lim_{n\to\infty} (TrA^n))^{\frac{1}{n}} = \lim_{n\to\infty} (\lambda_1^n + \cdots \lambda_m^n)^{\frac{1}{n}} = \lim_{n\to\infty} \sqrt[n]{\lambda_1^n \left[1 + (\frac{\lambda_2}{\lambda_1})^n + \cdots + (\frac{\lambda_m}{\lambda_1})^n\right]} = |\lambda_1|$$

2.2 Dimostrazione della seconda formula

Essendo A ortogonalmente diagonalizzabile per il teorema spettrale, e considerando λ_1 l'autovalore massimo senza perdere di generalità si ha:

$$\lim_{n\to\infty} \frac{Tr(A^{n+1})}{Tr(A^n)} = \lim_{n\to\infty} \frac{Tr\left[(PDP^T)^{n+1}\right]}{Tr\left[(PDP^T)^n\right]} = \lim_{n\to\infty} \frac{Tr(PD^{n+1}P^T)}{Tr(PD^nP^T)} = \lim_{n\to\infty} \frac{Tr(D^{n+1})}{Tr(D^n)} =$$

$$= \lim_{n\to\infty} \frac{\lambda_1^{n+1} + \cdots + \lambda_m^{n+1}}{\lambda_1^n + \cdots + \lambda_m^n} = \lim_{n\to\infty} \frac{\lambda_1^{n+1}\left[1 + (\frac{\lambda_2}{\lambda_1})^{n+1} + \cdots + (\frac{\lambda_m}{\lambda_1})^{n+1}\right]}{\lambda_1^n\left[1 + (\frac{\lambda_2}{\lambda_1})^n + \cdots + (\frac{\lambda_m}{\lambda_1})^n\right]} = \lambda_1$$

3 Implementazione in linguaggio Fortran delle formule

Già nelle prime fasi dell'implementazione si prende in considerazione la possibilità di utilizzare un kind selezionato per una precisione maggiore. Infatti,si dovrà calcolare la potenza di una matrice, quindi è plausibile che si otterranno valori molto elevati per la traccia.
Sono state create alcune funzioni utili all'implementazione dell'algoritmo, contenute nel `module calcolo_matriciale`.
Per il codice delle funzioni si guardi il contenuto del modulo nel file `massimo_autovalore.f90`

3.1 Potenza di una matrice

Il primo problema che ci si pone davanti è calcolare la potenza di una matrice. Procediamo dunque a creare una funzione che calcoli la potenza n-esima di una matrice quadrata.

La funzione creata è la seguente:

```
function mat_pow (A, n) result(C)
```

dove A è una matrice quadrata di dimensione qualsiasi a valori reali e n è l'esponente della potenza che si vuole calcolare. La funzione dà in output una matrice C della stessa dimensione di A.

L'algoritmo funziona in questo modo:

- Condizione di controllo dell'esponente: se l'esponente è pari a 1, si pone C=A e la funzione si arresta. Infatti per come è definito il seguito della funzione, con n=1 si bloccherebbe e non sarebbe in grado di calcolare la potenza.
- Si definisce un ciclo do (do k=1, n-1) che contiene l'algoritmo per calcolare il prodotto tra matrici C=B*A. Ovviamente è necessario utilizzare una matrice temporanea B che viene uguagliata a C (B=C) alla fine di ogni iterazione.

3.2 Calcolo della traccia di una matrice

La seguente funzione permette il calcolo della traccia di una matrice:

```
function tr (A) result(t)
```

dove A è una matrice quadrata di dimensione qualunque a valori reali e t contiene il valore real della traccia. L'algoritmo è tale da sommare con un ciclo do (do i=1,n) i valori A(i,i) presenti sulla diagonale della matrice.

3.3 Algoritmo di controllo che la matrice sia simmetrica

Le dimostrazioni svolte nella prima sezione ci permettono di concludere che l'algoritmo è funzionante per qualunque matrice diagonalizzabile. Tuttavia il quesito chiede di lavorare con matrici simmetriche, si inserisce dunque all'inizio dell'algoritmo il codice per controllare che la matrice di partenza sia effettivamente simmetrica.

La seguente funzione è stata inserita nel `module calcolo_matriciale`:

```
function sym(A) result(simmetria)
```

Dove A è una matrice quadrata di dimensione qualsiasi a valori reali e ***simmetria*** è una variabile di tipo logical che viene inizializzata a .TRUE. Se la matrice non è quadrata si assegna il valore .FALSE. altrimenti vengono controllate ogni coppia di elementi della matrice sopra e sotto alla diagonale e nel momento in cui si trovano due valori diversi si assegna il valore .FALSE. e si termina l'esecuzione della funzione.

Il valore di ***simmetria*** viene controllato all'inizio dell'esecuzione del programma principale, dopo l'inserimento da file della matrice.

3.4 Implementazione delle due formule

Utilizzando le funzioni appena introdotte è possibile scrivere le due formule come:

```
max_autoval_1=tr(mat_pow(A,n))**(1.0_rk/n)
max_autoval_2=tr(mat_pow(A,n+1))/tr(mat_pow(A,n))
```

4 Analisi del comportamento in funzione di N

I valori vengono confrontati con una prima stima fornita da *Wolfram Alpha*: $\lambda_W = 4.46050487$ I risultati seguenti sono stati ottenuti con **rk=selected_real_kind(6)** e la matrice A del testo del quesito.

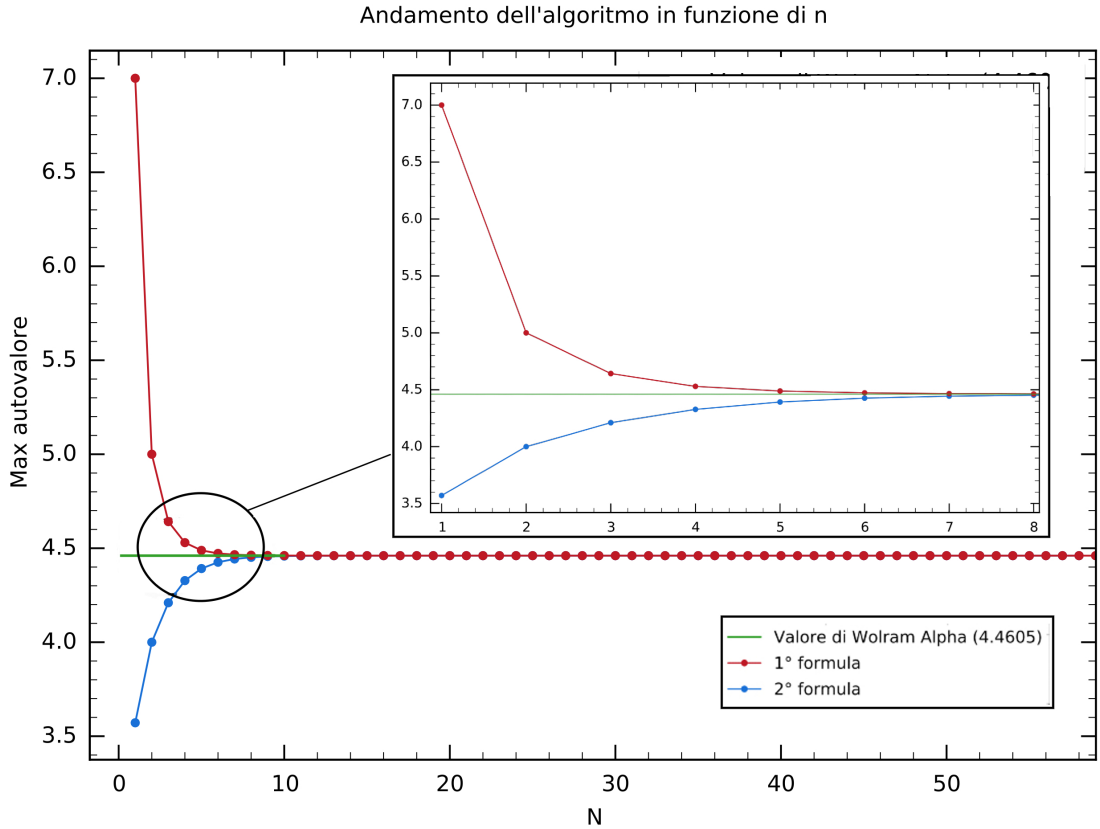

Figura 1: Andamento dei risultati delle formule in funzione di N, con zoom per $N \in [1, 8]$

Entrambe le formule convergono molto velocemente: già con N=10 sono molto vicini al valore λ_W ($|\lambda_W - \lambda_1| \simeq 10^{-3}$). Il valore di entrambi gli algoritmi rimane poi stabile, con minime variazioni. E' interessante notare come il primo algoritmo converga al valore vero dall'alto mentre il secondo dal basso. Più avanti si troverà che questo comportamento dipende dalla matrice di partenza.

4.1 Comportamento dello scarto rispetto al valore atteso

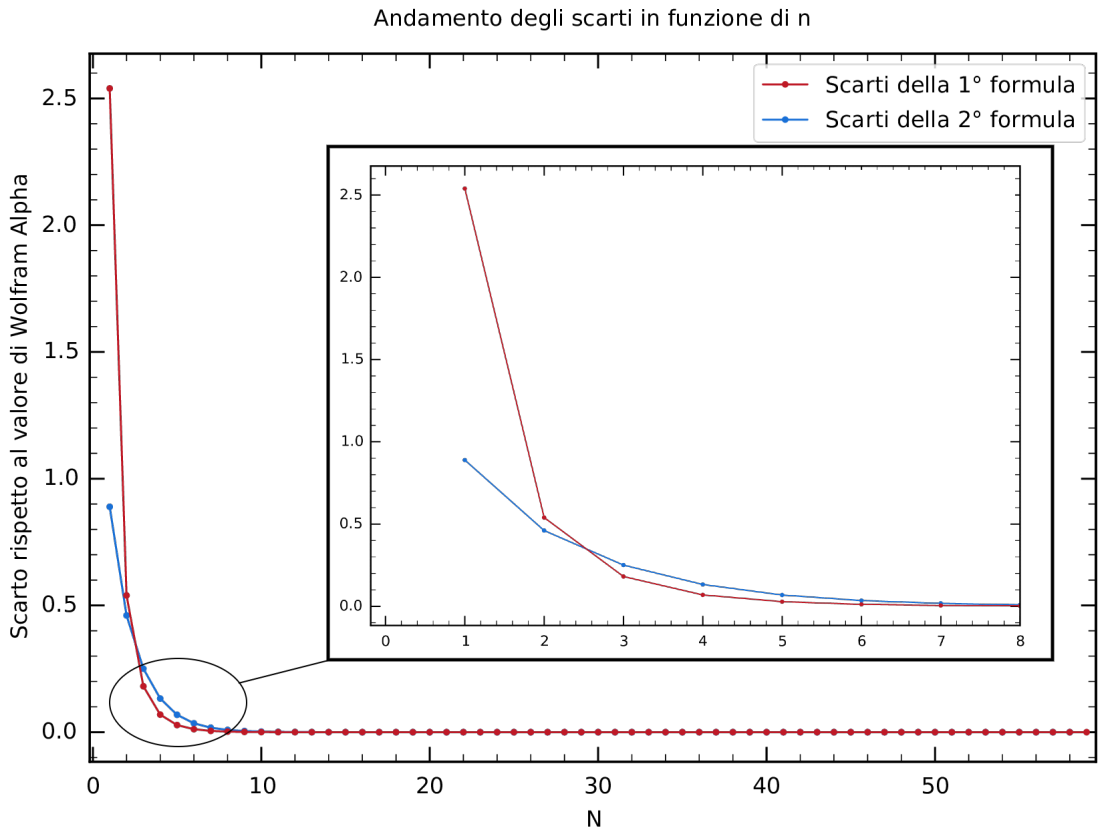

Figura 2: Andamento degli scarti in funzione di n, con zoom per $n \in [1, 8]$

Sembra effettivamente convergere più velocemente la prima formula. Nonostante ciò, per valori di $N > 10$, per entrambe le formule si ha $|\lambda_W - \lambda_1| \lesssim 10^{-6}$.

Può essere interessante osservare particolari valori (*per chiarezza nella sequente tabella non sono state inserite tutte le cifre decimali degli scarti, in quanto sono significativi soltanti gli ordini di grandezza*):

N	λ_1 (1°)	$\lvert\lambda_W - \lambda_1\rvert$ (1°)	λ_1 (2°)	$\lvert\lambda_W - \lambda_1\rvert$ (2°)
1	7.00000000	2.54	3.57142854	0.89
5	4.48858738	2.81 E-02	4.39187717	6.86 E-02
10	4.46095800	4.53 E-04	4.45825005	2.25 E-03
20	4.46050501	0	4.46050262	2.38 E-06
30	4.46050501	0	4.46050501	0
40	4.46050501	0	4.46050453	4.77 E-07
50	4.46050453	4.77 E-07	4.46050501	0
60	Infinity	Infinity	NaN	NaN

Si può notare che la prima formula sembra convergere più velocemente della seconda. Infatti raggiunge scarti dell'ordine del 10^{-7} con valori vicini a N=20, mentre la seconda formula li raggiun-

ge con N=23. La differenza come si può dunque notare è minima. Ovviamente gli scarti indicati con 0, sono da considerarsi dell'ordine di 10^{-7} in quanto il kind utilizzato e la precisione del valore di Wolfram Alpha ne limitano la rappresentazione.

5 Comportamento per N ancora maggiori, e possibilità di scelta dei kind

Particolarmente interessante è l'ultima riga della tabella del paragrafo precedente. E' evidente che la scelta del kind ci pone dei limiti sul valore di N utilizzabile. Una matrice come quella del quesito, i quali valori diversi da zero sono maggiori o uguali di 1, comporta che continuando a fare potenze sempre maggiori, si ottengano matrici con valori sempre più grandi, si arriverà dunque a raggiungere il limite imposto dalla scelta del kind.

A questo proposito, osserviamo l'andamento in funzione di N per kind maggiori.

Per rk=selected_real_kind(15) si cominciano ad avere dei problemi (infinity e NaN) per N= 475
Per rk=selected_real_kind(33) si cominciano ad avere dei problemi per N=7596.
E' necessario richiedere a Wolfram Alpha una stima più precisa del valore per continuare a confrontare i risultati ottenuti dalle due formule:

$$\lambda_W = 4.460504870018763491979644788353662 64$$

Analizziamo qualche valore interessante (*per chiarezza nella tabella seguente non sono state inserite tutte le cifre degli scarti, in quanto è interessante soltanto l'ordine di grandezza*):

rk=selected_real_kind(15)		
N	λ_1 (1°)	$\lvert\lambda_W - \lambda_1\rvert$ (1°)
30	4.4605048701747485	1.55 E-010
170	4.4605048700187631	0 ($\lesssim 10^{-16}$)
400	4.4605048700187639	8.88 E-016
N	λ_1 (2°)	$\lvert\lambda_W - \lambda_1\rvert$ (2°)
30	4.4605048676882975	2.33 E-009
170	4.4605048700187631	0 ($\lesssim 10^{-16}$)
400	4.4605048700187631	0 ($\lesssim 10^{-16}$)
rk=selected_real_kind(33)		
N	λ_1 (1°)	$\lvert\lambda_W - \lambda_1\rvert$ (1°)
50	4.460504870018763588609953756173146 58	9.66 E-0017
80	4.460504870018763491979644851713346 71	6.34 E-0026
100	4.460504870018763491979644788353715 13	5.24 E-0032
7000	4.460504870018763491979644788353662 75	0 ($\lesssim 10^{-34}$)
N	λ_1 (2°)	$\lvert\lambda_W - \lambda_1\rvert$ (2°)
50	4.460504870018761085830612307801377 83	2.41 E-0015
80	4.460504870018763491979642264046870 38	2.52 E-0024
100	4.460504870018763491979644788351056 58	2.61 E-0030
7000	4.460504870018763491979644788353662 75	0 ($\lesssim 10^{-34}$)

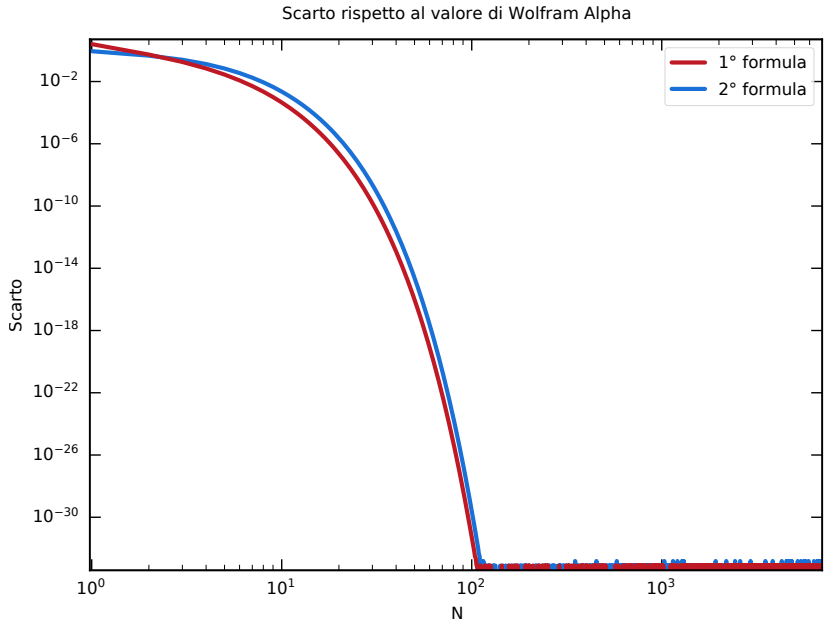

Figura 3: Andamento degli scarti rispetto al valore atteso λ_W in scala log-log

Osservando il grafico è evidente che utilizzando kind più precisi è possibile ottenere uno scarto decisamente inferiore, tuttavia non è necessario utilizzare valori di N molto elevati ($\gg 110$). Infatti con rk=selected_real_kind(33) e con N=112 si ottiene uno scarto $\lesssim 10^{-34}$ e questo è sostanzialmente lo scarto massimo raggiungibile con il kind selezionato.

Analizzando i dati ottenuti si è notato un risultato particolarmente interessante: per entrambe le formule con valori di $N \gtrsim 120$, i λ_i ottenuti si ripetono più volte, anche utilizzando valori di N molto differenti. Per esempio, la prima formula fornisce lo stesso risultato per N=128 e N=6995, mentre la seconda formula fornisce lo stesso risultato per N=130 e N=7000, lo stesso comportamento è stato osservato per molti altri valori. In conclusione non sembra esserci alcun guadagno in termini di precisione del risultato ottenuto nell'utilizzare valori di $N > 120$ per entrambe le formule, questo è dovuto alla velocità di convergenza molto elevata dell'algoritmo, sarà possibile fare una stima di questa nel paragrafo seguente.

5.1 Comportamento dell'errore relativo tra iterazioni successive

Si graficano (in scala log-log) i valori $\frac{|\lambda_n - \lambda_{n-1}|}{\lambda_{n-1}}$, ottenuti con rk=selected_real_kind(33):

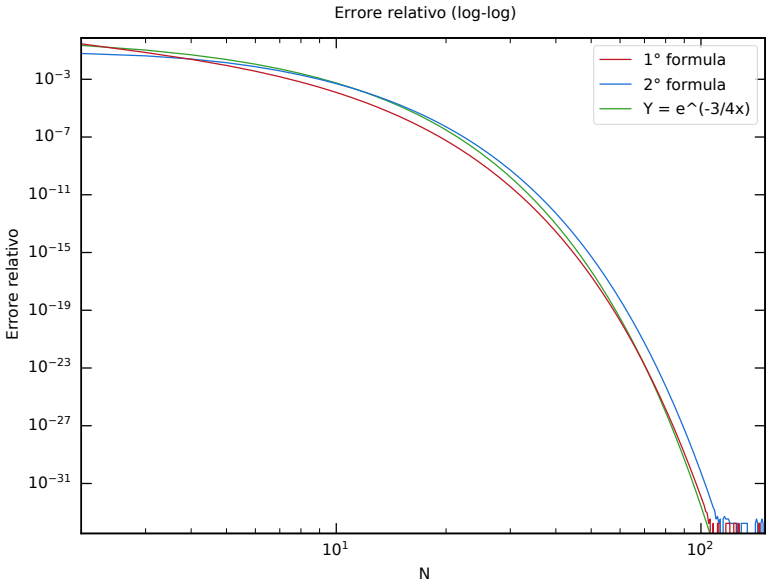

Figura 4: Andamento degli errori relativi, il grafico è in scala log-log

Il grafico conferma le conclusioni raggiunte nel paragrafo precedente.
Inoltre ci permette di fare una stima della velocità di convergenza delle due formule (che è simile da come si può vedere dal grafico). Possiamo affermare con certezza che gli errori relativi non convergono a zero come una qualsiasi potenza, se fosse questo il caso il grafico in scala log-log dovrebbe somigliare ad una retta. Per stimare la convergenza è utile a questo punto analizzare il grafico in scala semi-logaritmica:

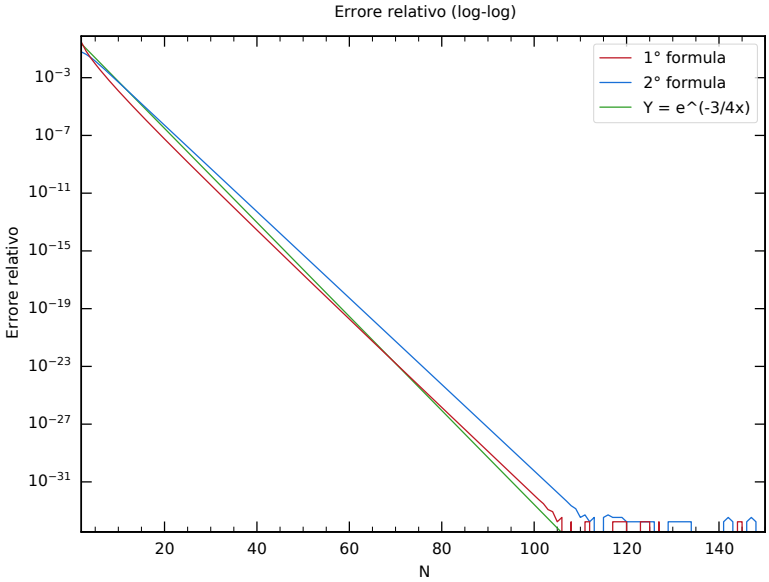

Figura 5: Andamento degli errori relativi, il grafico è in scala semi-log

In questo grafico l'andamento delle due formule assomiglia molto ad una retta, è dunque plau-

sibile che la convergenza segua un esponenziale del tipo $A \cdot e^{-B \cdot n}$, dove il valore di B è stimabile dalla pendenza della retta.

Applicando il metodo della regressione lineare (`regressione_lineare.f90`) ad entrambe le formule si ottengono le seguenti stime:

$$E_{rel\ 1} \sim \frac{1}{e^{0.7168 \cdot n - 2.2966}}$$

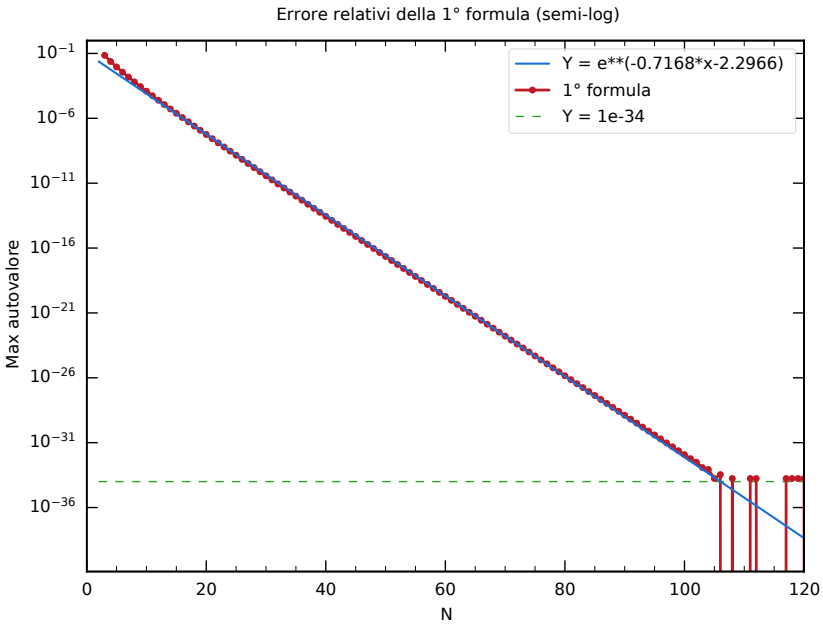

Figura 6: Regressione lineare degli errori relativi della prima formula

$$E_{rel\ 2} \sim \frac{1}{e^{0.6892 \cdot n - 0.7066}}$$

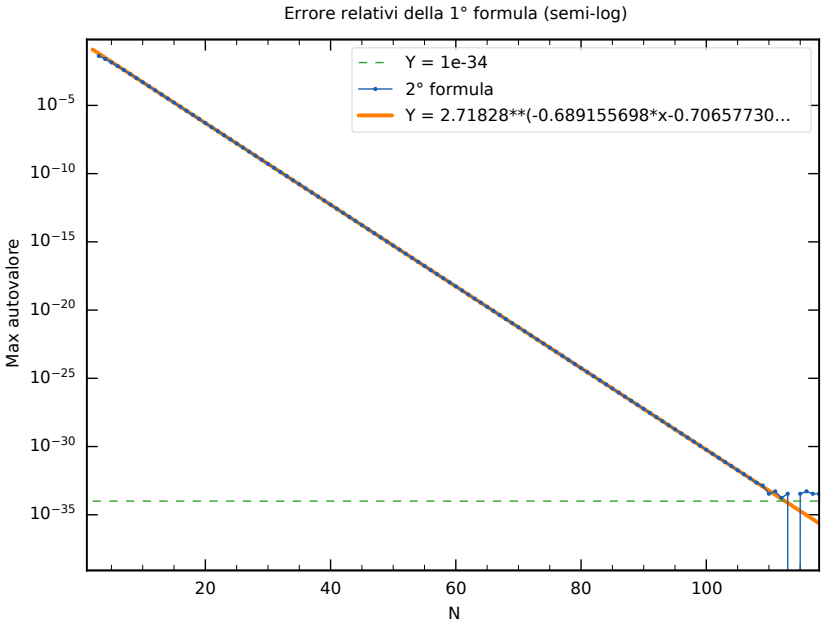

Figura 7: Regressione lineare degli errori relativi della seconda formula

Si ha inoltre la conferma che la prima formula converge più velocemente della seconda per questi valori di N.

6 Approfondimenti

6.1 Condizione d'uscita

Essendo la scelta del valore di N da utilizzare dipendente dai risultati che si vogliono ottenere, potrebbe risultare utile introdurre una condizione d'uscita per trovare il valore di N ottimale per i propri scopi.

- Si potrebbe uscire dal ciclo quando lo scarto tra il valore che si ottiene e quello del passo precedente è minore di una certa quantità ϵ, l' $\widetilde{N}$ ottimale risulterebbe in questo caso il primo valore per cui $\left|\lambda_{\widetilde{N}+1} - \lambda_{\widetilde{N}}\right| < \epsilon$

- Se si conosce il polinomio caratteristico della matrice (si può sempre calcolare con un algoritmo apposito), si può terminare l'esecuzione quando si trova un valore $\lambda_{\widetilde{N}}$ tale per cui $P(\lambda_{\widetilde{N}}) < \epsilon$. Nel caso della matrice A fornita dal quesito di partenza, il polinomio caratteristico è il seguente: $P(\lambda) = -\lambda^3 + 7\lambda^2 - 12\lambda + 3$, dalla teoria inoltre sappiamo che λ_1 è un autovalore se e solo se $P(\lambda_1) = 0$. Può essere interessante graficare l'andamento dei $P(\lambda_N)$ delle due formule:

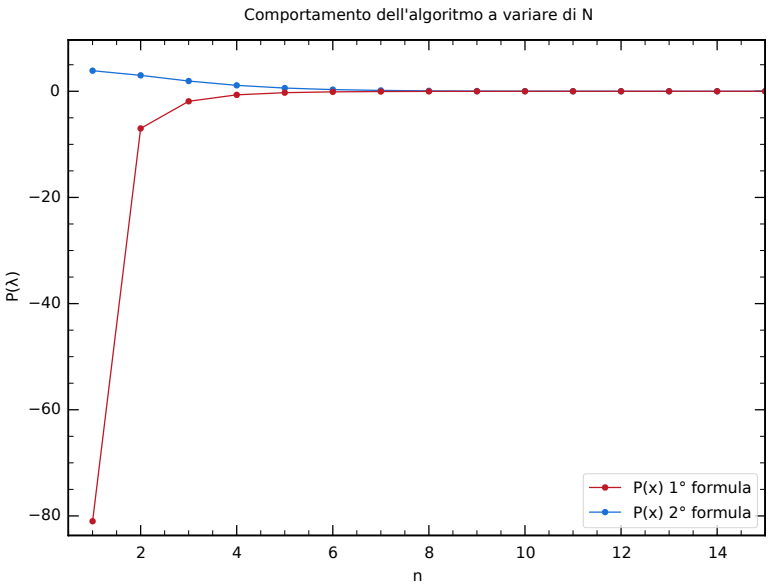

Figura 8: Andamento del valore di $P(\lambda_N)$

6.2 Casi particolari e limiti di funzionamento dell'algoritmo

Vi sono casi in cui la natura della matrice non pone particolari limiti sul valore di N utilizzabile. Infatti per esempio, considerando la matrice unità 3x3, essendo $\mathbb{I}^N = \mathbb{I} \quad \forall \quad N \in \mathbb{N}$, i valori della traccia rimangono costanti e non diventano sempre più grandi. Per come è definita la seconda formula, ci si aspetta che assuma proprio il valore $\lambda_1 = 1.0000000$ e rimanga costante su quel valore. Invece per la prima formula è necessario utilizzare valori di N molto grandi per ottenere uno scarto inferiore a 10^{-7} infatti con N=1e7 si ottiene $\lambda_1 = 1.00000011$, e con N=1e8 si ottiene $\lambda_1 = 1.000000011$.

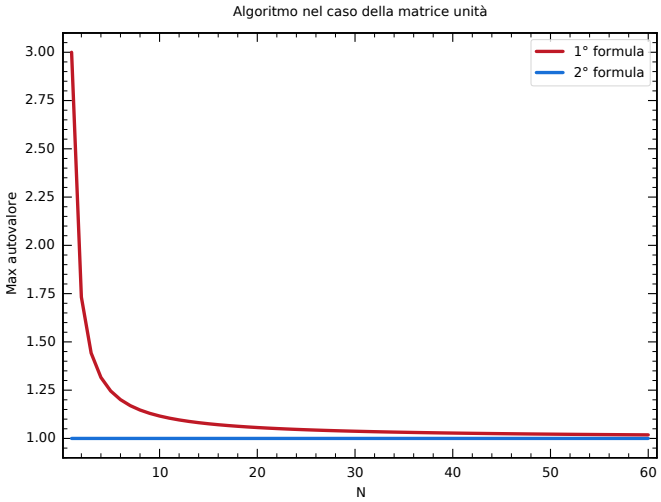

Figura 9: Algoritmo applicato alla matrice unità

Mentre se applichiamo l'algoritmo alla matrice:

$$A = \begin{pmatrix} -5 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

che ha -5 come autovalore massimo in modulo, si ottiene come risultato 5 dalla prima formula e -5 dalla seconda, come previsto dalla teoria.

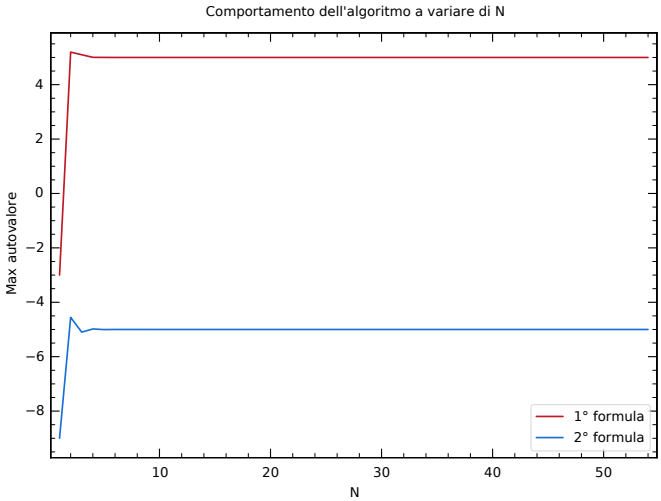

Figura 10

In questo caso la prima formula converge dal basso, dunque il comportamento osservato all'inizio dell'analisi dell'algoritmo non è generale, ma dipende dalla matrice di partenza.

6.3 Problemi dell'algoritmo

Se si esegue l'algoritmo (con rk=selected_real_kind(33)) con matrice di partenza

$$A = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

i risultati della prima formula si alternano tra 0 e valori decrescenti che si avvicinano sempre di più a 1, che è effettivamente l'autovalore massimo in modulo. La seconda formula invece alterna 0 con infinity. E' evidente che ci sia qualche problema con questa particolare matrice.

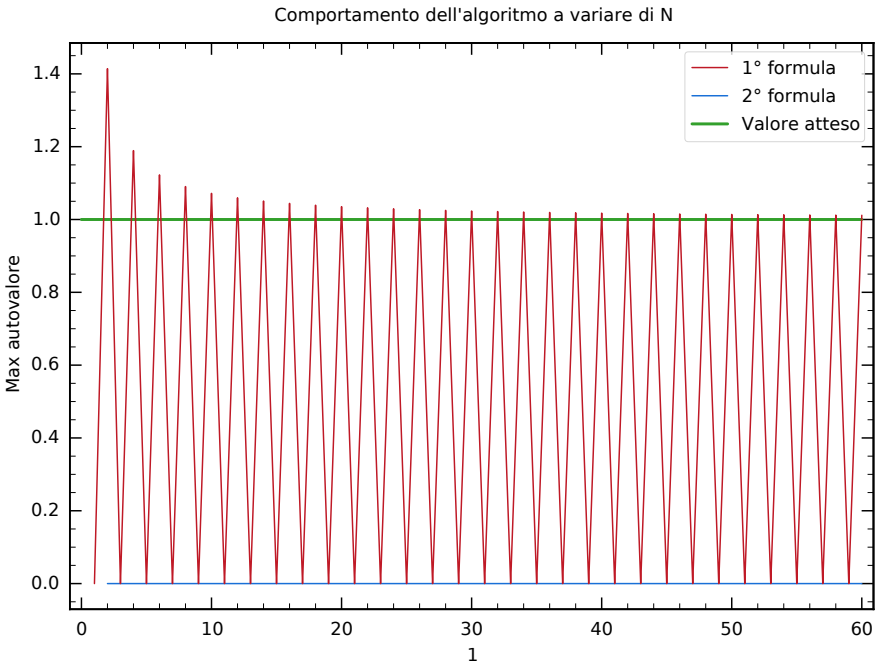

Figura 11: Algoritmo applicato alla matrice definita precedentemente

Bisogna in questo caso notare che la matrice simmetrica A, presenta effettivamente **l'autovalore massimo in modulo con entrambi i segni**, la seconda formula dovrebbe dunque convergere ad uno dei due, ma dalla dimostrazione precedente, non è chiaro a quale. Questa potrebbe essere la causa dei problemi dell'algoritmo. Effettuando altri test con matrici con questa proprietà, sono stati trovati comportamenti anomali ancora differenti, in cui la seconda formula oscilla tra valori sempre più vicini a zero e valori sempre più grandi. ***La presenza nello spettro della matrice dell'autovalore massimo in modulo con segni opposti crea problemi ad entrambe le formule, in particolare alla seconda.***

Per risolvere parzialmente il problema, si possono utilizzare soltanto valori di N pari. Il risultato è il seguente:

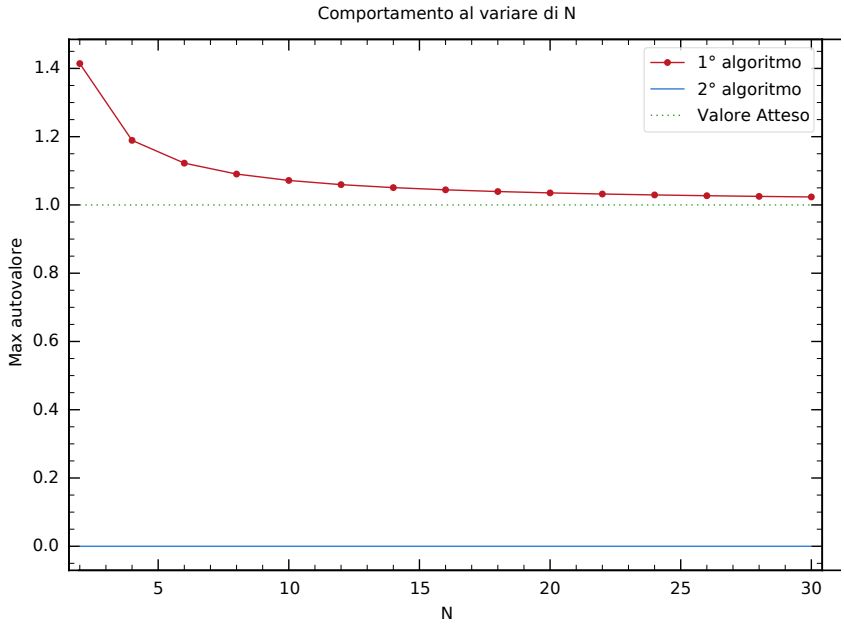

Figura 12: Andamento con soltanto valori di N pari

La prima formula ora converge senza oscillare, tuttavia la velocità di convergenza è piuttosto minore rispetto al caso con matrici non problematiche, infatti con N=30 si ottiene uno scarto $|\lambda_1 - \lambda_W| \simeq 2.3 \ \times 10^{-2}$ e un errore relativo $|\lambda_{N=30} - \lambda_{N=28}| \simeq 1.65 \times 10^{-3}$.
La seconda formula continua a dare problemi, tuttavia non alterna più valori nulli con infinity, ma assume il valore costante nullo.